

Python

Les Classes en Python

Introduction à la Programmation Orientée Objet (POO)

01 Pourquoi les classes ?

Limites des fonctions • Concept d'objet • Analogie mathématique

Les limites de la programmation avec fonctions seules

Imagine que tu gères plusieurs étudiants dans un programme. Avec des fonctions seules, les données sont éparpillées et difficiles à relier ensemble.

Approche avec fonctions — problématique ❌

```
# Les données sont éparpillées dans des
# variables séparées... difficile à gérer !

nom1 = 'Alice'
age1 = 20
notes1 = [15, 18, 12]

nom2 = 'Bob'
age2 = 22
notes2 = [10, 14, 16]

def calculer_moyenne(notes):
    return sum(notes) / len(notes)

# Pour 100 étudiants ? Cauchemar !
# nom1, nom2, nom3, ... nom100 ???
```

Ce qu'on voudrait ✔

```
# Regrouper données + comportements
# dans un seul endroit → une CLASSE

etudiant1 = Etudiant('Alice', 20)
etudiant1.ajouter_note(15)
etudiant1.ajouter_note(18)

etudiant2 = Etudiant('Bob', 22)
etudiant2.ajouter_note(10)

# Calculer la moyenne de Alice :
print(etudiant1.moyenne())

# Pour 100 étudiants ? Facile :
etudiants = [Etudiant(...) for ...]
```

💡 Une classe regroupe des données (attributs) et des fonctions (méthodes) qui vont ensemble. C'est comme créer un nouveau TYPE de variable.

Le concept d'objet — données + comportements

En POO : une CLASSE est un moule (plan), un OBJET est ce qu'on fabrique avec ce moule. On peut créer autant d'objets qu'on veut à partir d'une même classe.



Classe

Le plan de construction. Définit la structure et le comportement.
Ex : le plan d'une voiture.



Objet (instance)

Un exemplaire créé à partir de la classe.
Ex : la voiture rouge de Paul.



Attribut

Une donnée appartenant à l'objet.
Ex : couleur, marque, vitesse.



Méthode

Une fonction appartenant à l'objet.
Ex : demarrer(), freiner().

🔗 Un int, un str, une liste — ce sont déjà des objets en Python ! Les classes te permettent de créer tes propres types.

Analogie mathématique — des objets que vous connaissez

En mathématiques, vous manipulez déjà des 'objets' avec des propriétés et des opérations. Les classes Python permettent de les représenter directement !

Fraction ($\mathbb{Q}$)

Attributs : numérateur, dénominateur

Méthodes : simplifier(), addition(), multiplication()

```
f = Fraction(3, 4) # représente 3/4
```

Vecteur 2D ($\mathbb{R}^2$)

Attributs : x, y

Méthodes : norme(), addition(), produit_scalaire()

```
v = Vecteur(3.0, 4.0) # → norme = 5.0
```

Matrice 2×2

Attributs : a, b, c, d (coefficients)

Méthodes : determinant(), transposee(), trace()

```
m = Matrice(1,2,3,4) # [[1,2],[3,4]]
```

Polynôme

Attributs : coefficients (liste)

Méthodes : evaluer(x), degre(), derivee()

```
p = Polynome([1,0,-1]) #  $x^2 - 1$ 
```

02 Définir une classe

class • __init__ • self • Attributs • Méthodes • Instanciation

Syntaxe de base — class et __init__

Structure minimale d'une classe

```
class NomDeLaClasse:
    """ Description de la classe (docstring) """

    def __init__(self, param1, param2):
        """ Constructeur : s'exécute à la création de l'objet """
        self.param1 = param1    # attribut d'instance
        self.param2 = param2    # attribut d'instance

    def ma_methode(self):
        """ Une méthode de la classe """
        return self.param1
```

Exemple concret — classe Etudiant

```
class Etudiant:
    def __init__(self, nom, age):
        self.nom = nom
        self.age = age
        self.notes = []

    def ajouter_note(self, note):
        self.notes.append(note)
```

Règles de nommage

Élément	Convention	Exemple
Classe	Majuscule initiale	Etudiant, Vecteur, Fraction
Attribut	snake_case	self.nom, self.nb_notes
Méthode	snake_case + verbe	calculer(), ajouter()
__init__	Toujours ce nom	Constructeur unique

self — à quoi ça sert ?

🔑 self désigne l'objet lui-même. C'est grâce à self que chaque objet garde SES propres données. Sans self, toutes les instances partageraient les mêmes valeurs !

self en action

```
class Etudiant:
    def __init__(self, nom, age):
        # self = cet étudiant précis
        self.nom = nom    # son nom à LUI
        self.age = age    # son âge à LUI

    def se_presenter(self):
        # self.nom → le nom de CET étudiant
        return f'Je m'appelle {self.nom}'
        # et j'ai {self.age} ans.'

# Créer deux étudiants
e1 = Etudiant('Alice', 20)
e2 = Etudiant('Bob', 22)

print(e1.se_presenter())
# → Je m'appelle Alice
print(e2.se_presenter())
# → Je m'appelle Bob
```

Ce qui se passe vraiment

```
# Quand tu écris :
e1.se_presenter()

# Python traduit ça en :
Etudiant.se_presenter(e1)
# → self = e1 ici

# Donc self.nom = e1.nom = 'Alice'
```

self est toujours 1er paramètre

Toutes les méthodes reçoivent self en premier.

self n'est pas un mot-clé Python

Par convention on l'appelle self, mais c'est juste un nom.

Ne pas oublier self.

self.nom ≠ nom. Sans self., c'est une variable locale !

Attributs et méthodes — exemple complet

Classe Etudiant complète

```
class Etudiant:
    def __init__(self, nom, age):
        self.nom = nom
        self.age = age
        self.notes = [] # liste vide

    def ajouter_note(self, note):
        if 0 <= note <= 20:
            self.notes.append(note)
        else:
            raise ValueError('Note invalide')

    def moyenne(self):
        if len(self.notes) == 0:
            return 0.0
        return sum(self.notes)/len(self.notes)

    def est_admis(self):
        return self.moyenne() >= 10.0

    def nb_notes(self):
        return len(self.notes)
```

Utilisation

```
# Créer un étudiant
alice = Etudiant('Alice', 20)

# Ajouter des notes
alice.ajouter_note(15)
alice.ajouter_note(12)
alice.ajouter_note(18)

# Accéder aux attributs
print(alice.nom)      # Alice
print(alice.age)      # 20
print(alice.notes)    # [15, 12, 18]

# Appeler les méthodes
print(alice.moyenne()) # 15.0
print(alice.est_admis()) # True
print(alice.nb_notes()) # 3
```

🔑 Attribut = variable de l'objet (alice.nom). Méthode = fonction de l'objet (alice.moyenne()). On y accède avec le point '.'

Instanciation — créer des objets

Instancier = créer un objet à partir d'une classe. Chaque objet créé est indépendant et possède ses propres attributs.

Créer et utiliser plusieurs objets

```
# Créer 3 étudiants (3 instances de la classe Etudiant)
alice = Etudiant('Alice', 20)
bob   = Etudiant('Bob', 22)
clara = Etudiant('Clara', 21)

# Chaque objet est INDÉPENDANT
alice.ajouter_note(15)
bob.ajouter_note(8)
clara.ajouter_note(18)

# Vérifier qui est admis
for etudiant in [alice, bob, clara]:
    statut = 'Admis' if etudiant.est_admis() else 'Recalé'
    print(f'{etudiant.nom} : {statut} (moy = {etudiant.moyenne()})')

# Résultat :
# Alice : Admis (moy = 15.0)
# Bob   : Recalé (moy = 8.0)
# Clara : Admis (moy = 18.0)
```

💡 La classe Etudiant est le moule. alice, bob, clara sont 3 objets fabriqués avec ce moule. Modifier alice.notes ne change pas bob.notes.

03 Méthodes spéciales

`__str__` • `__repr__` • `__eq__` • `__add__` • Opérateurs

__str__ et __repr__ — afficher un objet

Par défaut, `print(alice)` affiche `<__main__.Etudiant object at 0x7f...>` — pas très utile ! Les méthodes spéciales `__str__` et `__repr__` corrigent ça.

Sans méthodes spéciales ❌

```
alice = Etudiant('Alice', 20)
print(alice)
# <__main__.Etudiant object at 0x7f3a>
# → Pas lisible du tout !
```

Avec __str__ et __repr__ ✔

```
alice = Etudiant('Alice', 20)
print(alice)
# Etudiant(nom='Alice', age=20)
# → Clair et lisible !
```

Implémenter __str__ et __repr__

```
class Etudiant:
    def __init__(self, nom, age):
        self.nom = nom
        self.age = age
        self.notes = []

    def __str__(self):
        """ Appelé par print() — pour l'utilisateur """
        moy = self.moyenne() if self.notes else 'Pas de notes'
        return f'Étudiant : {self.nom} (âge: {self.age}) — Moy: {moy}'

    def __repr__(self):
        """ Appelé dans la console — pour le développeur """
        return f"Etudiant(nom='{self.nom}', age={self.age})"

alice = Etudiant('Alice', 20)
alice.ajouter_note(15)
print(alice)    # Étudiant : Alice (âge: 20) — Moy: 15.0
alice          # Etudiant(nom='Alice', age=20) ← dans la console
```

`__eq__`, `__add__` — opérer sur des objets

Les méthodes spéciales permettent d'utiliser les opérateurs `+`, `==`, `>`, `<` etc. sur vos objets. Très utile pour les objets mathématiques !

Classe `Vecteur2D` avec opérateurs

```
import math

class Vecteur2D:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f'({self.x}, {self.y})'

    def __eq__(self, autre):
        """ v1 == v2 """
        return self.x == autre.x and self.y == autre.y

    def __add__(self, autre):
        """ v1 + v2 """
        return Vecteur2D(self.x + autre.x,
                          self.y + autre.y)

    def __mul__(self, scalaire):
        """ v * 3 """
        return Vecteur2D(self.x * scalaire,
                          self.y * scalaire)

    def norme(self):
        return math.sqrt(self.x**2 + self.y**2)

    def produit_scalaire(self, autre):
        return self.x*autre.x + self.y*autre.y
```

Utilisation

```
v1 = Vecteur2D(3, 4)
v2 = Vecteur2D(1, 2)

print(v1)          # (3, 4)
print(v1.norme())  # 5.0

# Addition
v3 = v1 + v2
print(v3)          # (4, 6)

# Multiplication scalaire
v4 = v1 * 2
print(v4)          # (6, 8)

# Égalité
print(v1 == v2)    # False
print(v1 == Vecteur2D(3,4)) # True

# Produit scalaire
print(v1.produit_scalaire(v2))
# 3*1 + 4*2 = 11
```

Tableau des méthodes spéciales utiles

Ces méthodes commencent et finissent par `__` (double underscore = 'dunder'). Python les appelle automatiquement selon l'opération effectuée.

Méthode	Déclenchée par	Exemple d'usage
<code>__init__(self, ...)</code>	Constructeur	<code>obj = MaClasse(...)</code>
<code>__str__(self)</code>	<code>print(obj)</code> , <code>str(obj)</code>	<code>print(vecteur)</code>
<code>__repr__(self)</code>	Affichage console	<code>vecteur</code> ← dans REPL
<code>__eq__(self, other)</code>	<code>obj1 == obj2</code>	<code>v1 == v2</code>
<code>__lt__(self, other)</code>	<code>obj1 < obj2</code>	<code>f1 < f2</code> (Fraction)
<code>__add__(self, other)</code>	<code>obj1 + obj2</code>	<code>v1 + v2</code>
<code>__sub__(self, other)</code>	<code>obj1 - obj2</code>	<code>v1 - v2</code>
<code>__mul__(self, other)</code>	<code>obj1 * obj2</code>	<code>v1 * 3</code>
<code>__len__(self)</code>	<code>len(obj)</code>	<code>len(polynome)</code>
<code>__contains__(self, item)</code>	<code>item in obj</code>	<code>3 in ensemble</code>

04 Attributs & Encapsulation

Classe vs Instance • @property • _ et __

Attributs de classe vs attributs d'instance

Un attribut d'instance appartient à UN objet précis (self.nom). Un attribut de classe est partagé par TOUS les objets de la classe.

Exemple — attribut de classe

```
class Etudiant:
    # Attribut de CLASSE – partagé par tous
    universite = 'UQAM'
    nb_etudiants = 0 # compteur global

    def __init__(self, nom, age):
        # Attributs d'INSTANCE – propres à chaque
        # objet
        self.nom = nom
        self.age = age
        self.notes = []
        # Incrémenter le compteur global
        Etudiant.nb_etudiants += 1

    def __str__(self):
        return f'{self.nom} ({self.universite})'
```

Accès et différences

```
alice = Etudiant('Alice', 20)
bob   = Etudiant('Bob', 22)

# Attributs d'instance : propres à chaque objet
print(alice.nom)    # Alice
print(bob.nom)      # Bob

# Attribut de classe : partagé par tous
print(alice.universite) # UQAM
print(bob.universite)  # UQAM
print(Etudiant.nb_etudiants) # 2

# Modifier l'attribut de classe
Etudiant.universite = 'Concordia'
print(alice.universite) # Concordia
print(bob.universite)  # Concordia
```

Type	Déclaration	Portée
Instance	<code>self.nom = ...</code>	Un seul objet
Classe	<code>universite = ...</code>	Tous les objets

Encapsulation — protéger les données

L'encapsulation consiste à contrôler l'accès aux données d'un objet. On utilise `_` (convention) ou `__` (vraiment privé) pour indiquer qu'un attribut est interne.

Convention de nommage

```
class CompteBancaire:
    def __init__(self, titulaire, solde):
        self.titulaire = titulaire # public
        self._solde = solde        # _ = protégé
        # convention : 'ne pas modifier
        directement'

    # @property : accès contrôlé en lecture
    @property
    def solde(self):
        return self._solde

    # @solde.setter : modifier avec vérification
    @solde.setter
    def solde(self, valeur):
        if valeur < 0:
            raise ValueError('Solde négatif
interdit')
        self._solde = valeur

    def deposer(self, montant):
        if montant > 0:
            self._solde += montant

    def retirer(self, montant):
        if montant > self._solde:
            raise ValueError('Solde insuffisant')
        self._solde -= montant
```

Utilisation de @property

```
c = CompteBancaire('Alice', 1000)

# Lecture → passe par @property
print(c.solde)          # 1000

# Modification directe → déclenchée le setter
c.solde = 500           # OK
print(c.solde)          # 500

c.solde = -100          # ValueError !

# Utiliser les méthodes
c.deposer(200)
print(c.solde)          # 700

c.retirer(800)           # OK
print(c.solde)          # -100 → non,
                        # ValueError levée
```

⚠ `_` = 'je te fais confiance, ne modifie pas directement'.
`@property` = accès en lecture comme un attribut, mais avec du contrôle.

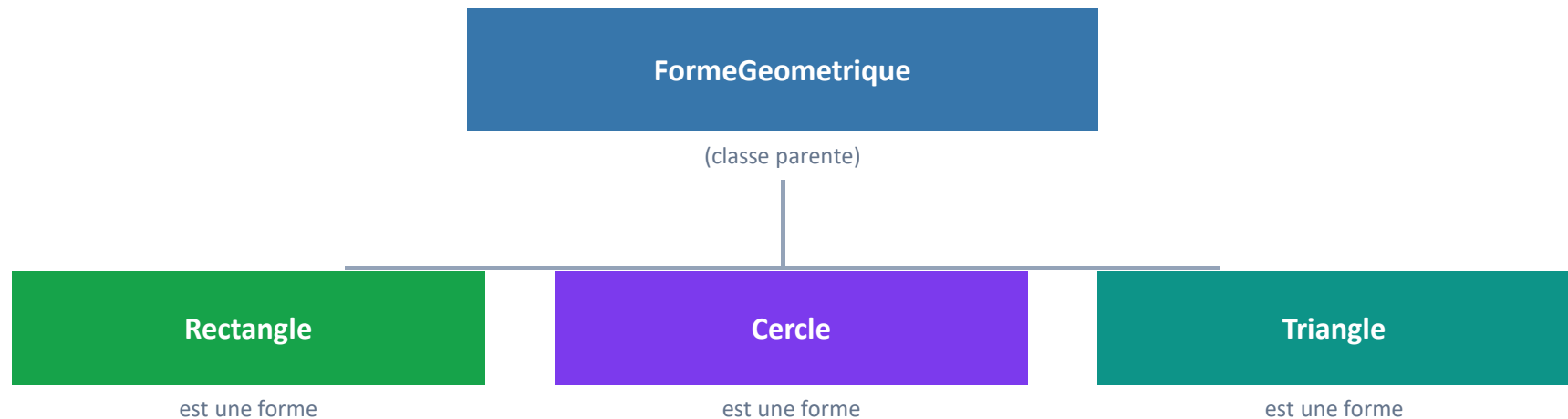
05 Héritage

Concept • Syntaxe • super() • Exemple géométrique

Héritage — réutiliser et spécialiser

L'héritage permet à une classe fille de récupérer tous les attributs et méthodes d'une classe parente, et d'y ajouter ses propres spécificités. Relation 'est un type de'.

Exemple — Hiérarchie de formes géométriques



Ce que chaque fille hérite automatiquement :

```
# FormeGeometrique (parente) définit :  
# - self.couleur (attribut)  
# - se_decrire() (méthode)  
  
# Rectangle, Cercle, Triangle héritent TOUT ça.  
# Chacun ajoute ce qui lui est propre :  
# - Rectangle : longueur, largeur, aire()  
# - Cercle : rayon, aire(), perimetre()  
# - Triangle : base, hauteur, aire()
```

Syntaxe de l'héritage et super()

Classe parente

```
class FormeGeometrique:
    def __init__(self, couleur):
        self.couleur = couleur

    def se_decrire(self):
        return f'Forme de couleur {self.couleur}'

    def aire(self):
        raise NotImplementedError(
            'Chaque forme doit définir aire()'
        )
```

Utilisation

```
r = Rectangle('rouge', 5, 3)
c = Cercle('bleu', 4)

# Méthode héritée du parent :
print(r.se_decrire()) # Forme de couleur rouge
print(c.se_decrire()) # Forme de couleur bleu

# Méthode propre à chaque fille :
print(r.aire())       # 15
print(c.aire())       # ~50.27
```

Classes filles — syntaxe héritage

```
# class NomFille(NomParente):
class Rectangle(FormeGeometrique):

    def __init__(self, couleur, longueur, largeur):
        # super() appelle __init__ du parent
        super().__init__(couleur)
        self.longueur = longueur
        self.largeur = largeur

    def aire(self):
        return self.longueur * self.largeur

    def perimetre(self):
        return 2 * (self.longueur + self.largeur)

class Cercle(FormeGeometrique):
    import math
    def __init__(self, couleur, rayon):
        super().__init__(couleur)
        self.rayon = rayon

    def aire(self):
        import math
        return math.pi * self.rayon ** 2
```

06 Exemples Complets

Fraction • Vecteur 2D • Bonnes pratiques

Exemple complet — Classe Fraction

fraction.py

```
from math import gcd

class Fraction:
    def __init__(self, num, den):
        if den == 0:
            raise ValueError('Dénominateur = 0 !')
        # Simplification automatique
        diviseur = gcd(abs(num), abs(den))
        signe = -1 if den < 0 else 1
        self.num = signe * num // diviseur
        self.den = signe * den // diviseur

    def __str__(self):
        if self.den == 1:
            return str(self.num)
        return f'{self.num}/{self.den}'

    def __repr__(self):
        return f'Fraction({self.num},{self.den})'

    def __add__(self, autre):
        """ a/b + c/d = (ad + bc) / bd """
        return Fraction(
            self.num*autre.den + autre.num*self.den,
            self.den * autre.den
        )

    def __mul__(self, autre):
        return Fraction(self.num*autre.num,
            self.den*autre.den)

    def __eq__(self, autre):
        return self.num==autre.num and
            self.den==autre.den

    def valeur(self):
        return self.num / self.den
```

Tests et utilisation

```
# Création
f1 = Fraction(3, 4)    # 3/4
f2 = Fraction(1, 2)    # 1/2
f3 = Fraction(6, 8)    # simplifié → 3/4

print(f1)    # 3/4
print(f2)    # 1/2
print(f3)    # 3/4 (automatiquement simplifié)

# Addition : 3/4 + 1/2 = 5/4
print(f1 + f2)    # 5/4

# Multiplication : 3/4 × 1/2 = 3/8
print(f1 * f2)    # 3/8

# Égalité (après simplification)
print(f1 == f3)    # True

# Valeur décimale
print(f1.valeur())    # 0.75

# Fractions incorrectes
# Fraction(5, 0) → ValueError

# Simplification automatique
print(Fraction(10, 4))    # 5/2
print(Fraction(-6, -4))    # 3/2
```

Exemple complet — Classe Vecteur2D

vecteur.py

```
import math

class Vecteur2D:
    def __init__(self, x, y):
        self.x = float(x)
        self.y = float(y)

    def __str__(self):
        return f'({self.x}, {self.y})'

    def __repr__(self):
        return f'Vecteur2D({self.x}, {self.y})'

    def __add__(self, autre):
        return Vecteur2D(self.x+autre.x,
                          self.y+autre.y)

    def __sub__(self, autre):
        return Vecteur2D(self.x-autre.x,
                          self.y-autre.y)

    def __mul__(self, scalaire):
        return Vecteur2D(self.x*scalaire,
                          self.y*scalaire)

    def __eq__(self, autre):
        return (math.isclose(self.x, autre.x) and
                math.isclose(self.y, autre.y))

    def norme(self):
        return math.sqrt(self.x**2 + self.y**2)

    def produit_scalaire(self, autre):
        return self.x*autre.x + self.y*autre.y

    def angle_avec(self, autre):
        """ Angle en degrés entre deux vecteurs """
        cos_ca = self.produit_scalaire(autre) / (self.norme() * autre.norme())
        angle = math.degrees(math.acos(cos_ca))
        return angle
```

Utilisation mathématique

```
v1 = Vecteur2D(3, 4)
v2 = Vecteur2D(1, 0)

# Opérations de base
print(v1 + v2)      # (4.0, 4.0)
print(v1 - v2)      # (2.0, 4.0)
print(v1 * 2)       # (6.0, 8.0)

# Propriétés
print(v1.norme())   # 5.0   (32+42=25)

# Produit scalaire
print(v1.produit_scalaire(v2)) # 3.0

# Angle entre les vecteurs
print(v1.angle_avec(v2))
# ~53.13°

# Vecteur normalisé
print(v1.normalise())
# (0.6, 0.8)

# Test d'orthogonalité
i = Vecteur2D(1, 0)
j = Vecteur2D(0, 1)
print(i.produit_scalaire(j)) # 0.0
print(i.angle_avec(j))      # 90.0°
```

Bonnes pratiques POO

1

Une classe = une responsabilité

Une classe doit faire UNE chose bien. Etudiant gère les étudiants, pas l'affichage, pas la BD.

2

Toujours écrire `__str__`

Chaque classe doit avoir `__str__` pour être lisible avec `print()`. Aide le débogage énormément.

3

Nommer clairement

Nom de classe en PascalCase (Vecteur2D). Attributs et méthodes en snake_case (produit_scalaire).

4

Valider dans `__init__`

Vérifier les paramètres à la création. Ex: `den != 0`, `montant > 0`. Lever `ValueError` si invalide.

5

Documenter avec docstrings

`"""Calcule la norme du vecteur."""` → indispensable pour les autres (et pour toi dans 6 mois !)

6

Tester avec pytest

Chaque méthode mérite un test. `test_addition()`, `test_str()`, `test_erreur()`. Code de qualité !

Résumé du cours



Classe & Objet

La classe est le moule (plan). L'objet est l'instance créée. Chaque objet a ses propres attributs.



Méthodes spéciales

`__str__`, `__repr__`, `__add__`, `__eq__`... permettent d'utiliser `print()`, `+`, `==` sur vos propres objets.



Héritage

`class Fille(Parente)` hérite tous les attributs et méthodes. `super()` appelle le constructeur parent.



self & `__init__`

`self` désigne l'objet lui-même. `__init__` est le constructeur appelé à la création de l'objet.



Encapsulation

`_` protège les données par convention. `@property` contrôle la lecture/écriture des attributs.



Exemples maths

Fraction, Vecteur2D : vos objets mathématiques avec `__add__`, `__mul__`, `norme()`, `produit_scalaire()`...

La POO transforme votre façon de penser le code : pensez objets, pas juste fonctions ! 🚀